

SESIÓN 01

Deep Learning

Introducción

Dr. León Felipe Palafox Novack

lfpalafox@up.edu.mx

01 Representaciones

02 Redes neuronales

03 Backpropagation

04 Redes convolucionales

05 Redes recurrentes

06 Transformers

02

Anuncios

AVISOS DE LA SESIÓN

Haz clic en una línea para escribir · se guarda en este navegador

01

02

03

03

Objetivos de la sesión

Al terminar la sesión, podrás:

01

Explicar

cómo una red aprende representaciones.

02

Distinguir

las funciones de la pérdida y del optimizador.

03

Implementar y validar

una red con PyTorch.

04

Metodología de aprendizaje

01

Teoría

Intuición

Mecanismo

Límites

02

Práctica

Implementar

Evaluar

Interpretar

03

Casos de uso

Fallas

Oportunidades

La clase combina una sección teórica y una práctica; en la práctica analizamos casos de uso de los algoritmos, sus fallas y sus oportunidades.

05

Mapa del curso



06

Evaluación

60%

Proyecto final

40%

Tareas y participación

Equipos de hasta 3 personas

Reporte de 3–5 páginas: datos, diseño y variables

Tareas

Participación en clase

07

Proyecto final

01 **Un problema concreto.**

02 **Un modelo.**

03 **Una evaluación.**

IDEAS DE PROYECTO

DEPORTES

Despeje óptimo del portero

¿A dónde patear para que su equipo gane el balón?

VISIÓN

Detección de autos

Localizar vehículos en video de calle

VISIÓN

Reconocimiento facial

Verificar identidad a partir de una foto

VISIÓN

Generación de arte

Transferir el estilo de una pintura a una foto

AUDIO

Generación de música

Componer secuencias nuevas

TEXTO

Generación de texto

Escribir con el estilo de un autor

TEXTO

Emojificador

Elegir el emoji adecuado para una frase

TEXTO

Traducción automática

Pasar un texto de un idioma a otro

AUDIO

Palabra de activación

Encender un sistema al oír una palabra

08

Contenido de esta sesión

1

Aplicaciones del aprendizaje profundo

AlphaFold y modelos de lenguaje

2

PyTorch y otros frameworks

Tensores, diferenciación automática y GPU

3

Fundamentos de redes neuronales

Perceptrón, activaciones y capas

4

Backpropagation y optimización

Regla de la cadena y gradiente descendente

5

CNN y RNN

Visión y secuencias

09

Aprendizaje profundo

¿Cómo aprende una red la representación que necesita?

10

El problema de clasificación

- 1 ¿Qué clase corresponde a cada observación?
- 2 ¿Qué limita a nuestro clasificador?
- 3 ¿Podemos cambiar la representación?

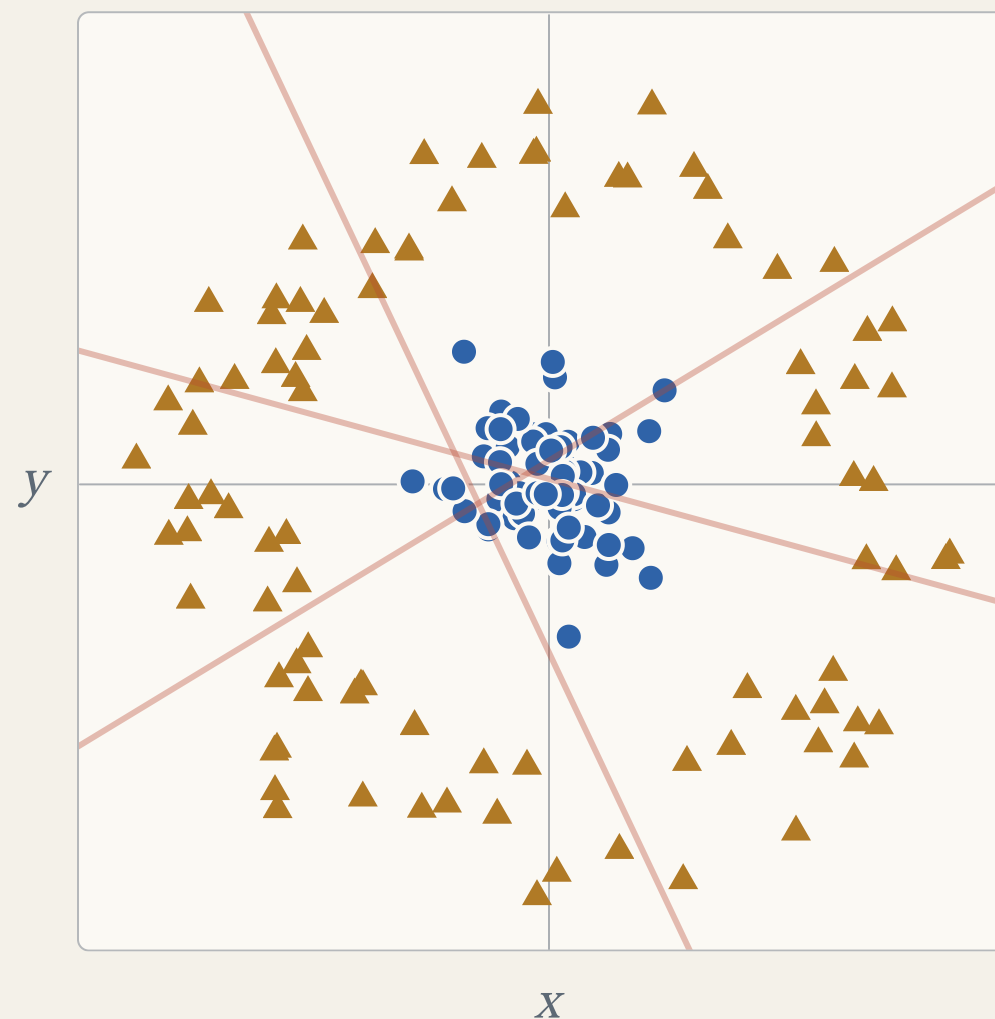
11

La representación cambia el problema

Una frontera lineal puede fallar en las coordenadas originales.

● Clase A ▲ Clase B

✗ Ninguna recta separa el centro del anillo.



12

El mismo dato, otras coordenadas

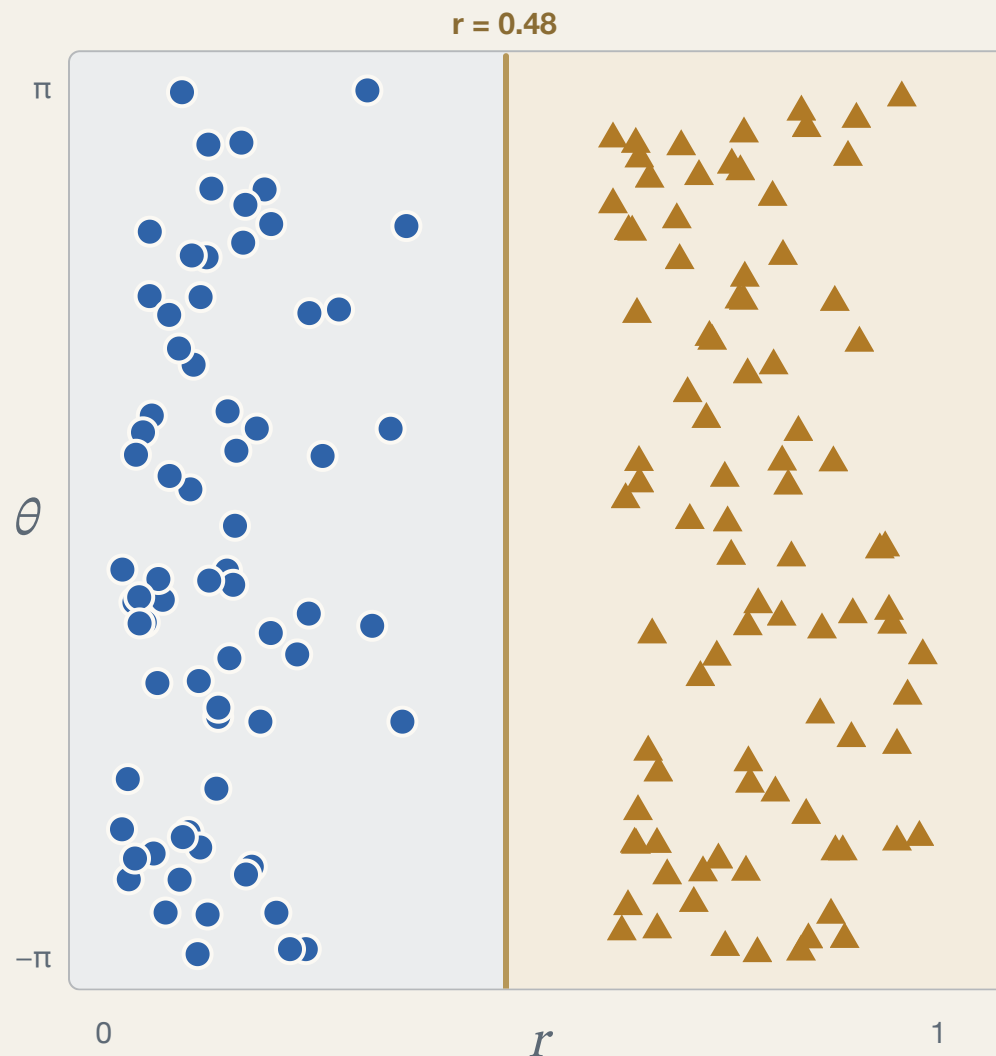
El radio puede separar clases que forman círculos concéntricos.

● Clase A ▲ Clase B

$$r = \sqrt{x^2 + y^2}$$

$$\theta = \text{atan2}(y, x)$$

✓ Una recta vertical, $r = 0.48$, separa las clases.



13

Clasificación y características

Las características describen el dato: píxeles, palabras o medidas.

Algunas clases son difíciles de separar con la representación inicial.

Transformar los datos puede simplificar la clasificación.

UNA OBSERVACIÓN → UN VECTOR DE CARACTERÍSTICAS

Imagen píxeles $x = [0.12, 0.80, 0.33, \dots]$

Texto conteo de palabras $x = [0, 2, 1, 0, \dots]$

Sensor medidas $x = [36.8, 72, 1.2, \dots]$

REPRESENTACIÓN INICIAL



DESPUÉS DE UNA TRANSFORMACIÓN $\phi(x)$

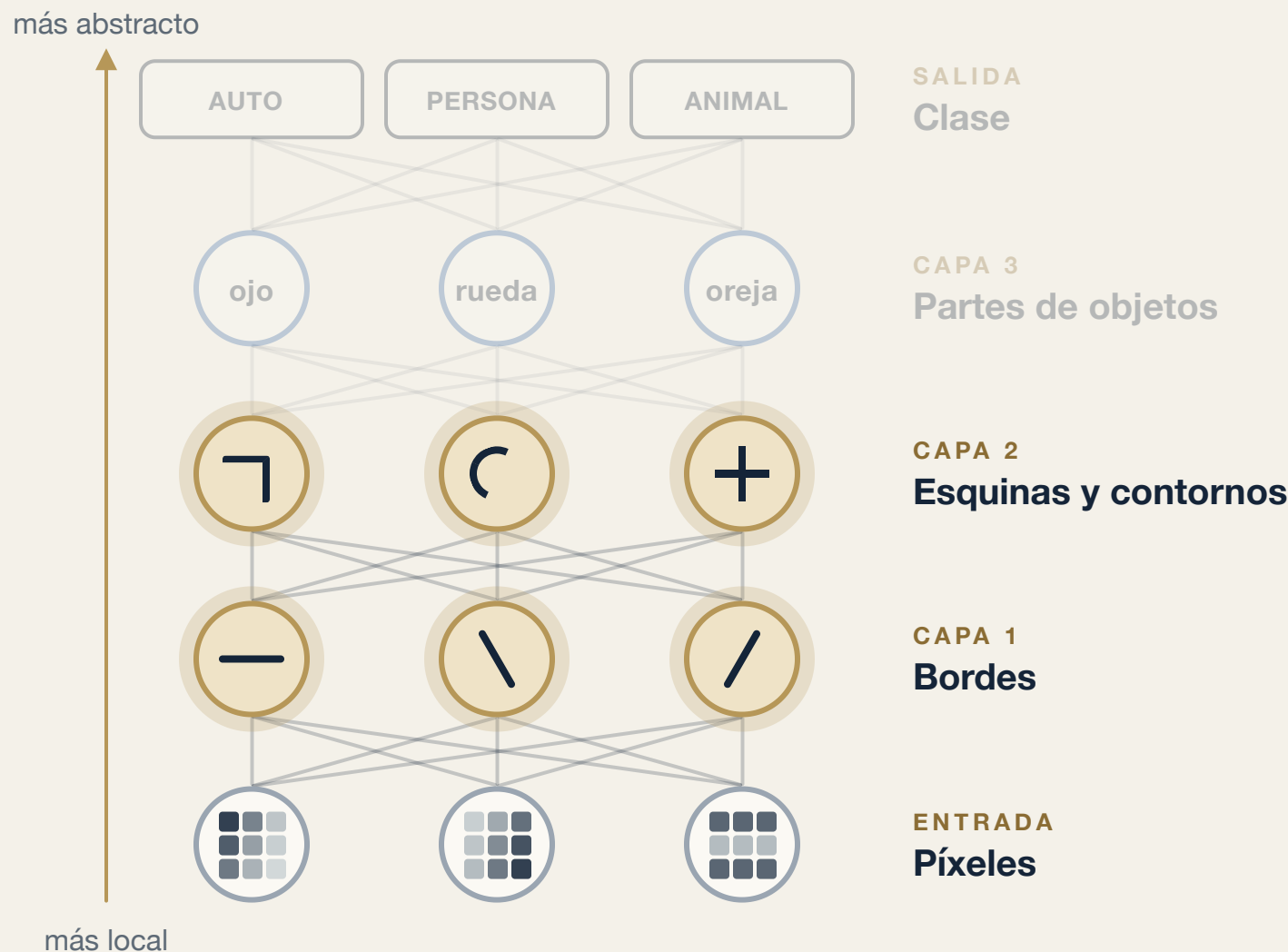


14

Capas de representación

Las primeras capas detectan patrones locales.

Bordes, esquinas y contornos: rasgos simples que aparecen en cualquier parte de la imagen.

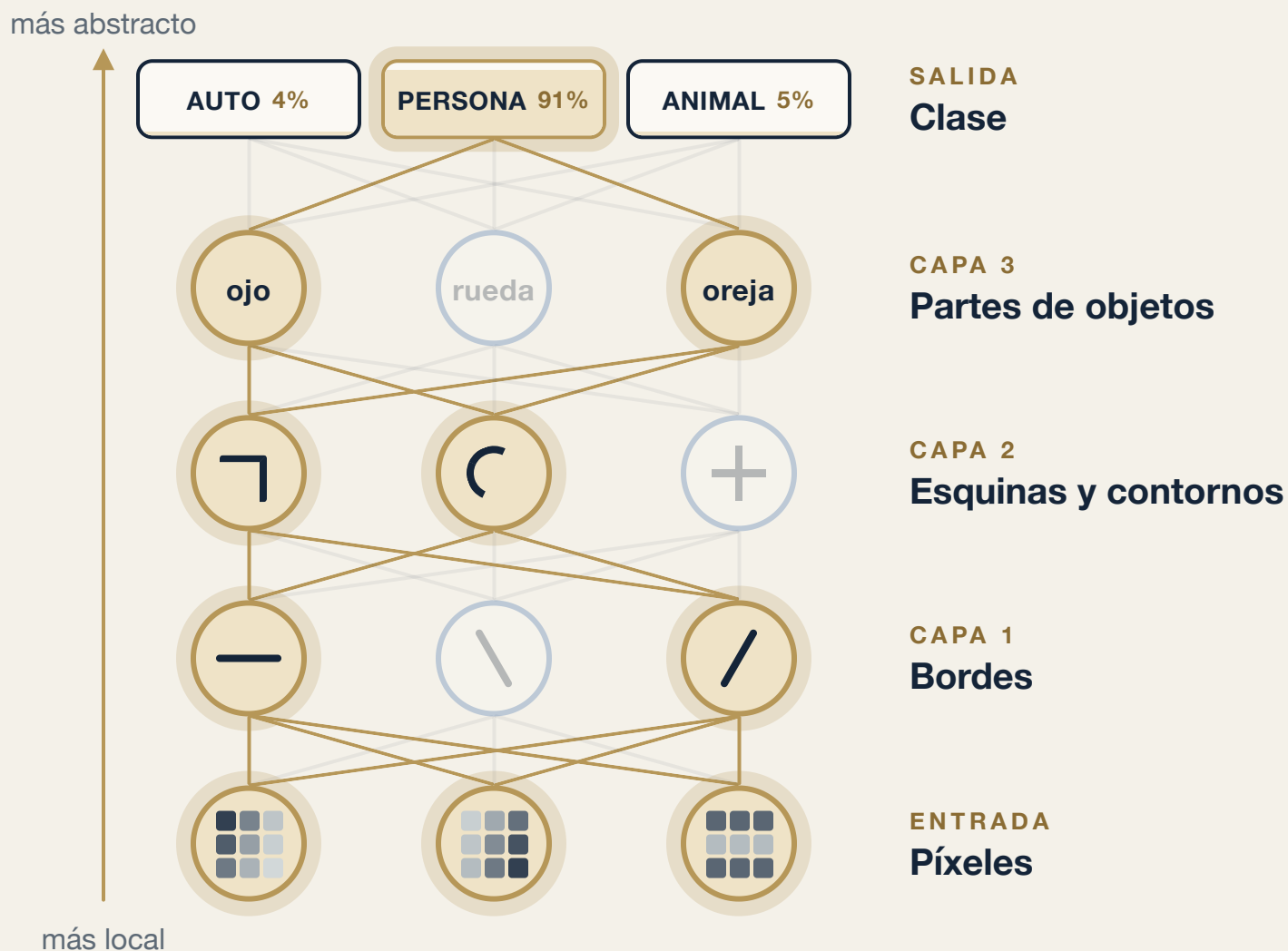


15

La combinación de patrones

Las capas combinan rasgos para identificar la clase.

Bordes → esquinas → partes (ojo, oreja) → **persona**.
Cada capa combina lo que detectó la anterior.



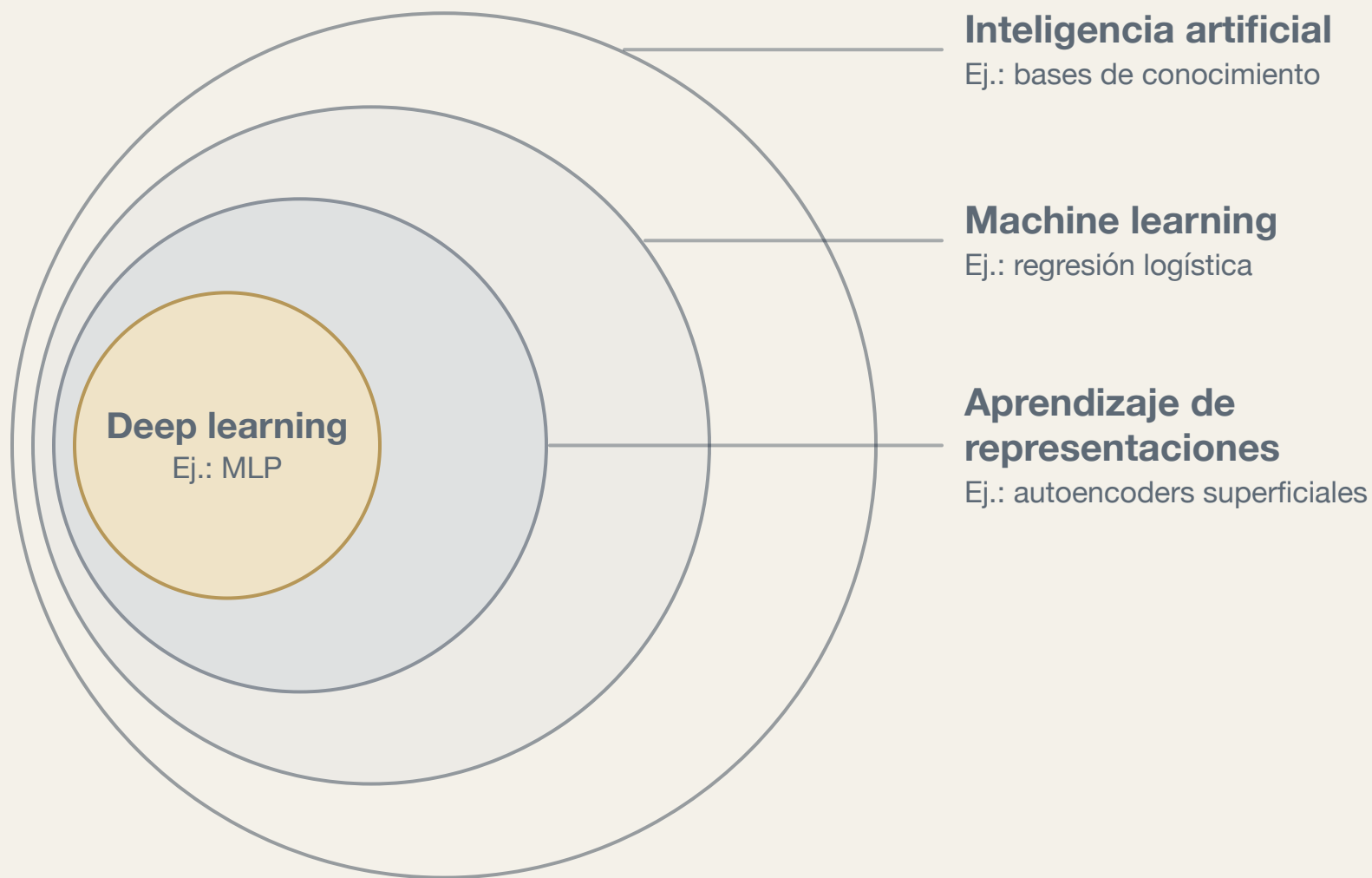
16

IA, machine learning y deep learning

Deep learning es una familia de métodos de machine learning.

$$DL \subset RL \subset ML \subset IA$$

Cada círculo agrega una idea: aprender de datos, aprender la representación y hacerlo con muchas capas.



17

Vocabulario de la clase

Redes neuronales profundas

Modelos con varias capas.

deep learning

redes profundas

redes neuronales

Neuronas o unidades

Transformaciones con parámetros:

$$h = f(w^T x + b)$$

Formas de aprendizaje

Supervisado

pares (x, y) etiquetados

Autosupervisado

la señal sale del propio dato

Por refuerzo

recompensas de un entorno

18

Prerrequisitos

1

Regresión logística

$$\hat{y} = \sigma(w^T x + b)$$

2

Gradiente descendente

$$w \leftarrow w - \eta \nabla L(w)$$

3

¿Qué función minimiza el optimizador?

$$L(w) = ?$$

19

Ventajas y límites

VENTAJAS

- + Aprender representaciones reduce el diseño manual de características.
- + Las GPU aceleran operaciones que pueden ejecutarse en paralelo.

LO QUE SIGUE SIENDO NECESARIO

Datos**Cómputo****Evaluación**

Datos, cómputo y evaluación siguen siendo necesarios: aprender la representación no elimina el trabajo de validar.

20

Herramientas para implementarlo

Python para programar.

Infraestructura local o en la nube.

LENGUAJE

`import torch`

FRAMEWORKS

PyTorch

TensorFlow

JAX

Keras

INFRAESTRUCTURA

LOCAL

laptop o estación de trabajo

NUBE

 Microsoft Azure  Google Colab

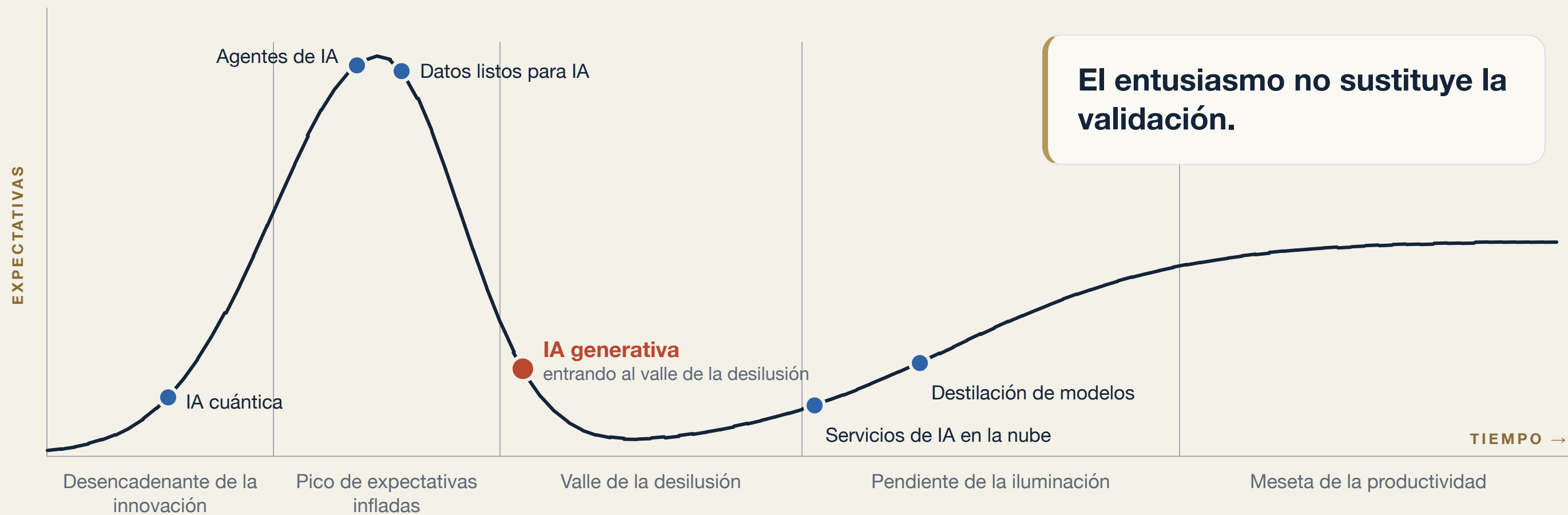
HARDWARE

CPU

GPU · CUDA

21

Expectativas y madurez



Posiciones según Gartner, *Hype Cycle for Artificial Intelligence, 2025* (junio de 2025). Selección de tecnologías.

22

GPU: el origen gráfico

Las GPU nacieron para dibujar, no para pensar.

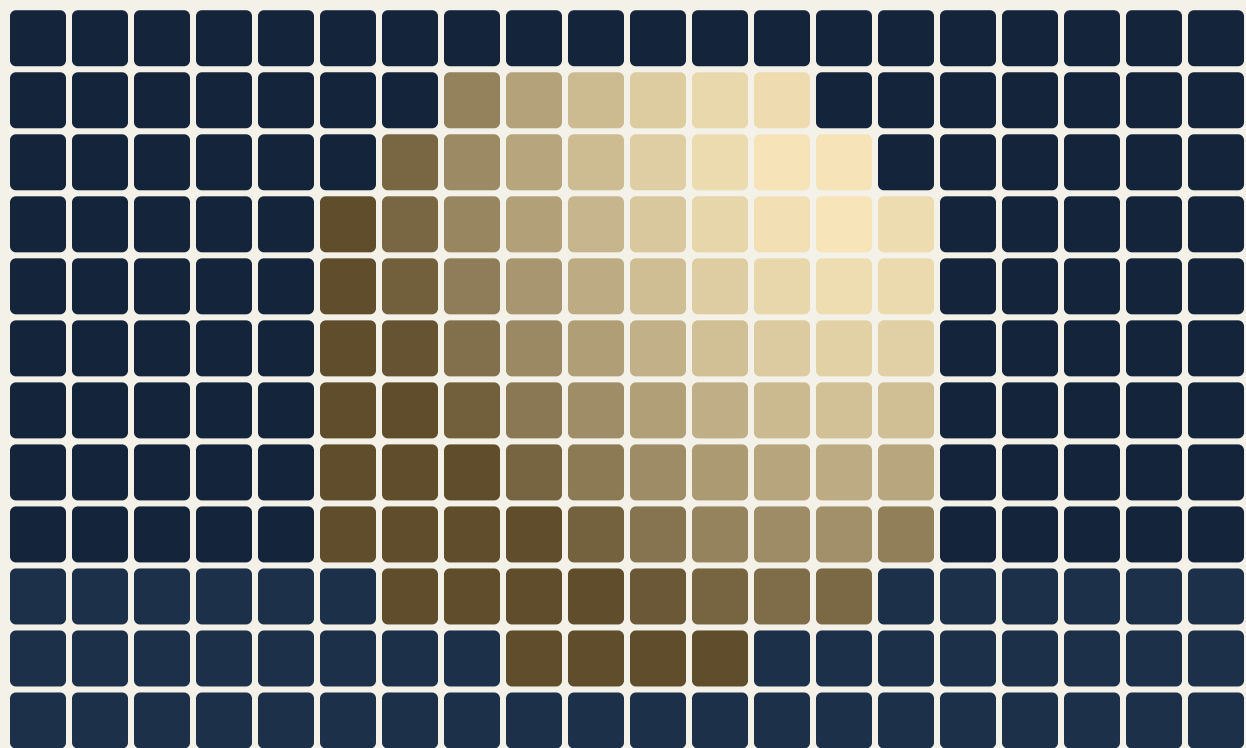
1993  **NVIDIA** se funda.

1998 3dfx Voodoo2 populariza la aceleración 3D en PC.

1999 GeForce 256: NVIDIA la presenta como «la primera GPU».

Nadie pensaba todavía en multiplicar matrices.

TRABAJO DE LA GPU: CALCULAR CADA PÍXEL, RÁPIDO



fotograma 3

Cada píxel se calcula por separado: miles de cálculos idénticos, en paralelo.

23

Cómputo paralelo y CUDA



NVIDIA convirtió un chip para videojuegos en un instrumento científico.

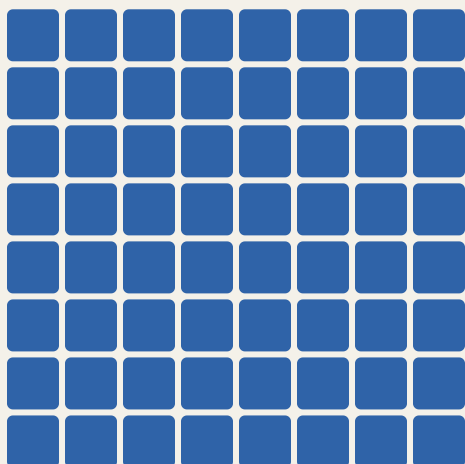
2006 GeForce 8800 GTX: 128 núcleos programables.

2007 CUDA: programar la GPU para cálculo general.

Ian Buck (de Stanford a NVIDIA) y John Nickolls impulsaron CUDA.

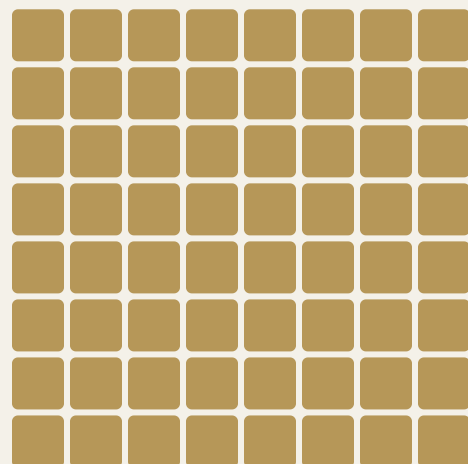
$C = A \times B$ · cada celda $C_{ij} = \sum_k A_{ik} B_{kj}$ es independiente

CPU · 4 núcleos



pasos: **16** ✓

GPU · 128 núcleos



pasos: **1** ✓

Misma operación: 16 pasos contra 1.

24

Datos, cómputo y redes profundas

Tres personas. Dos tarjetas gráficas. Un artículo.

EQUIPO

Alex Krizhevsky, Ilya Sutskever y Geoffrey Hinton



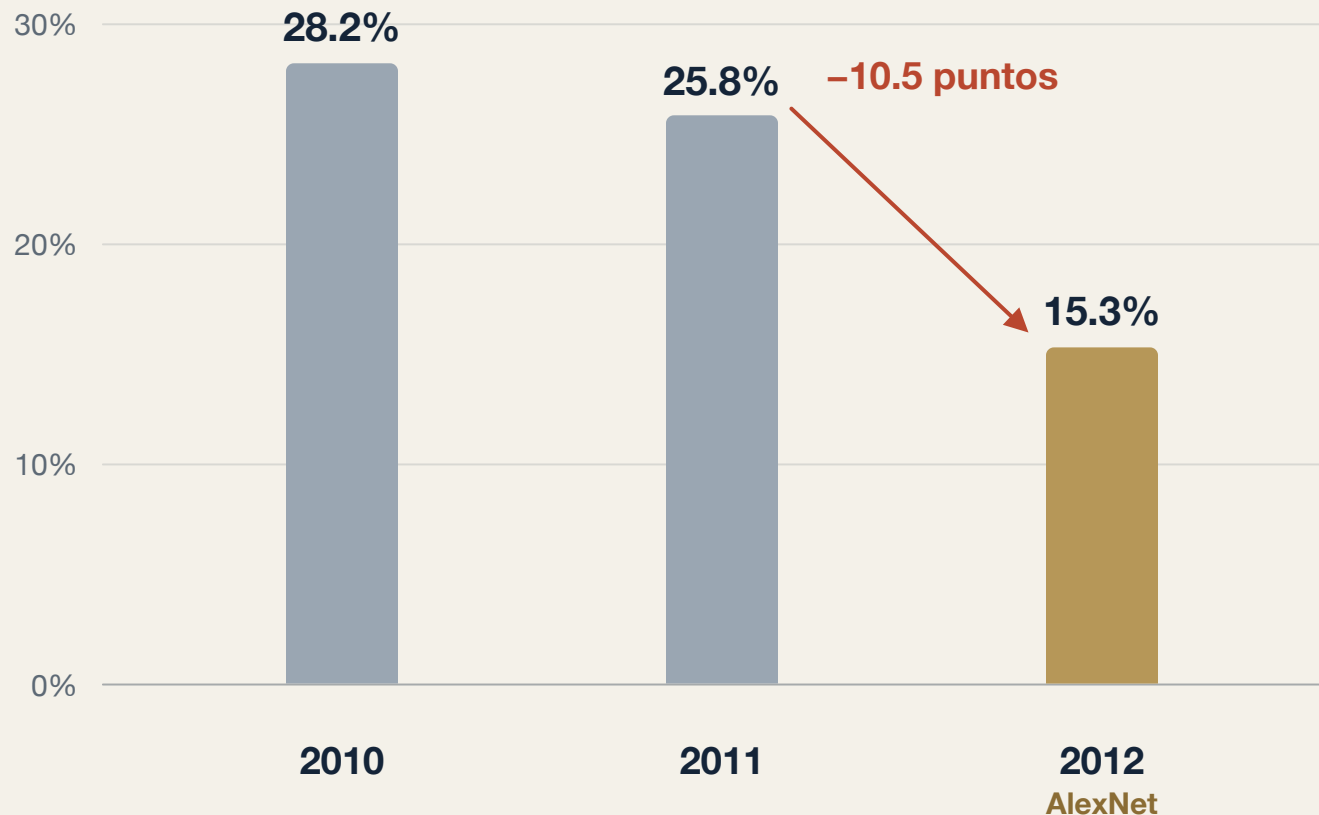
CÓMPUTO

2 GPU NVIDIA GTX 580 (3 GB) · 5–6 días de entrenamiento

ARTÍCULO

«ImageNet Classification with Deep Convolutional Neural Networks», NeurIPS 2012

ERROR TOP-5 DEL GANADOR DE ILSVRC (IMAGENET)



25

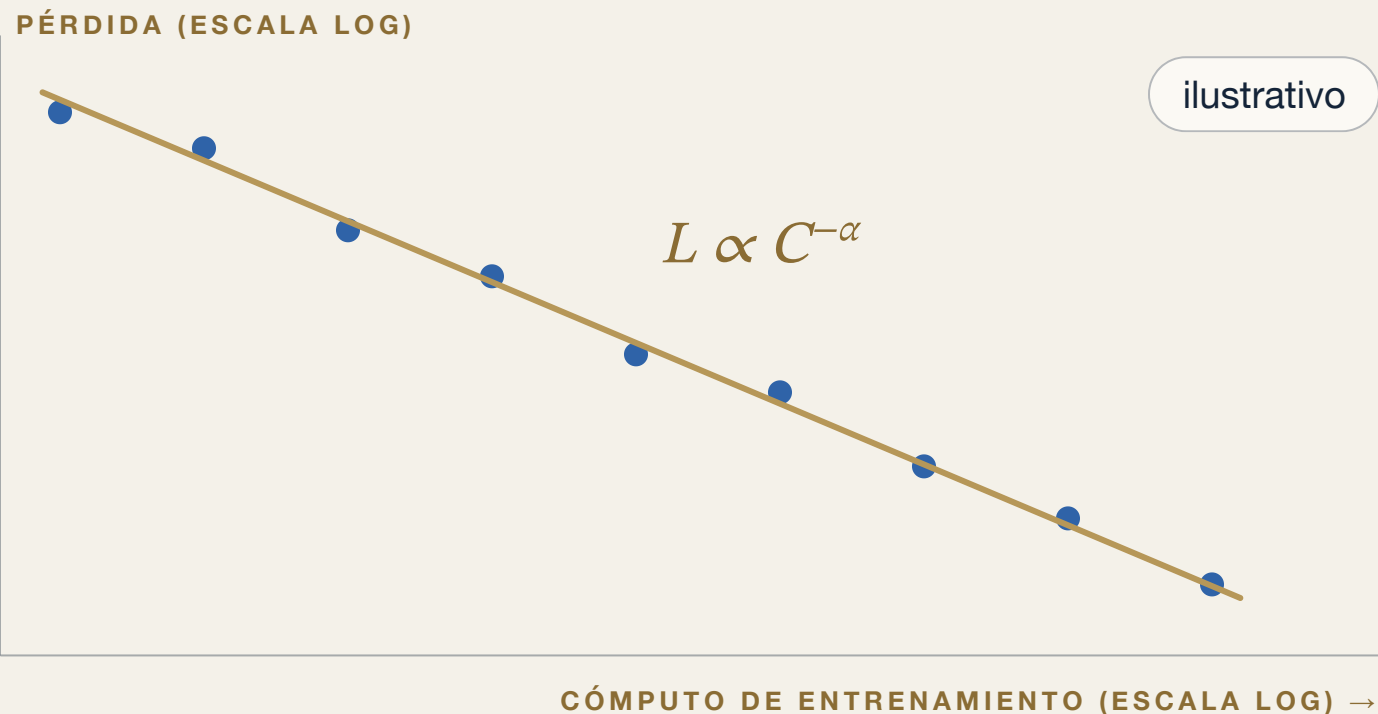
La escala del entrenamiento

Con las leyes de escalamiento, el cuello de botella pasó a ser el silicio.

KAPLAN ET AL., 2020

La pérdida de un modelo de lenguaje baja como una ley de potencia al crecer parámetros, datos y cómputo.

Más cómputo, mejores modelos de forma predecible: la carrera se volvió de GPU.



V100 · 2017



A100 · 2020



H100 · 2022

26

AlphaFold

Predicción de estructuras de proteínas.

AlphaFold destacó en CASP13. AlphaFold 2 destacó en CASP14.

AlphaFold 2 usa un sistema de redes basado en atención.

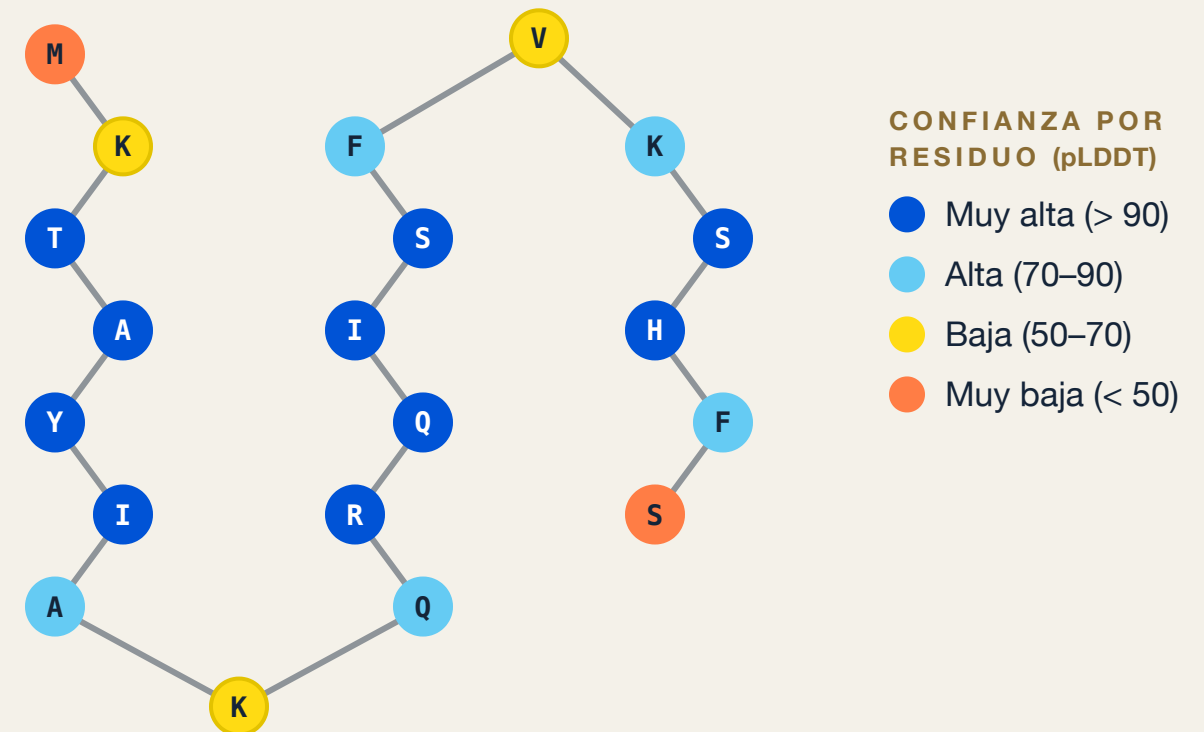
 **Google DeepMind**

CASP13 · 2018 AlphaFold

CASP14 · 2020 AlphaFold 2

2024 Nobel de Química: Hassabis y Jumper

ESTRUCTURA PREDICHA · TRES HÉLICES



27

Modelos de lenguaje

Los modelos Transformer aprenden patrones del lenguaje.

Generan texto a partir del contexto disponible.

Una respuesta plausible puede contener errores.

CONTEXTO

Las redes neuronales profundas aprenden representaciones jerárquicas y fueron inventadas en 2012.

SIGUIENTE PALABRA · PROBABILIDAD (ILUSTRATIVA)

jerárquicas		41%
útiles		30%
complejas		17%
a		7%



Plausible, pero falso. Las redes neuronales existen desde mediados del siglo XX; 2012 marcó su auge con AlexNet.

28

PyTorch

Framework de Python para construir y entrenar redes.

Tensoros, diferenciación automática y aceleración con GPU.

Otras opciones: TensorFlow, JAX y Keras.

TensorFlow

JAX

Keras

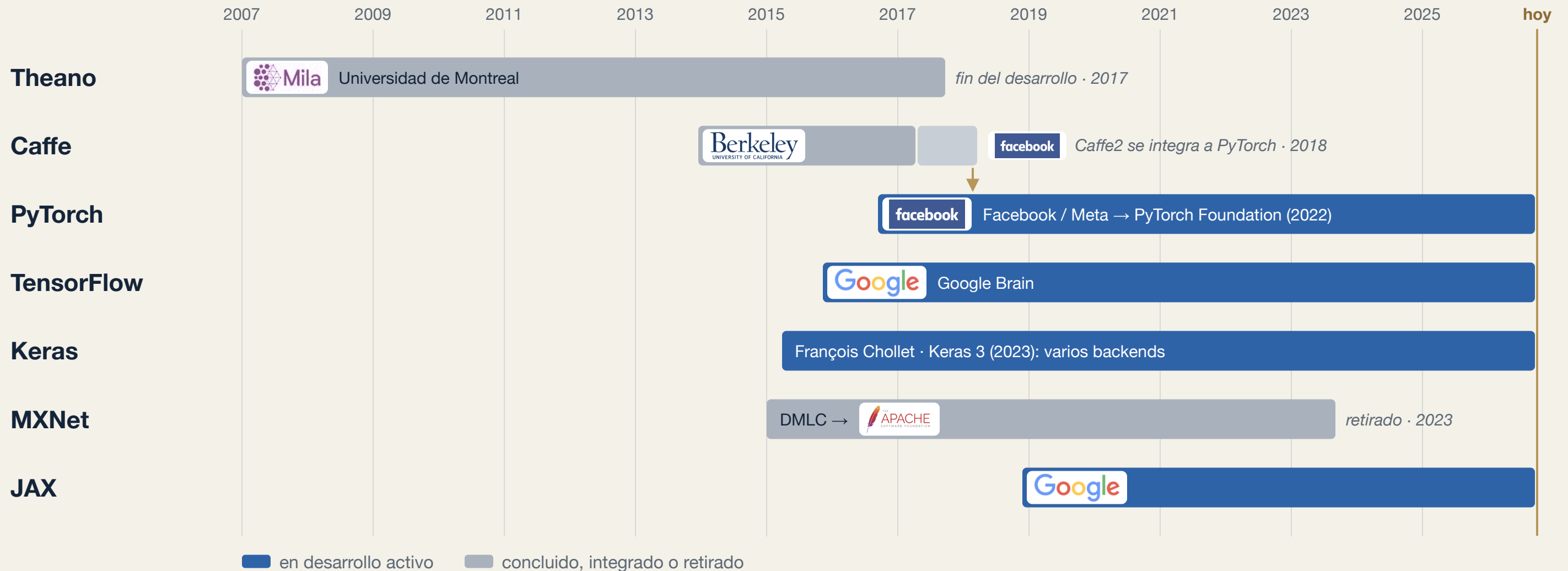
```
import torch
from torch import nn

# usa la GPU si está disponible
device = "cuda" if torch.cuda.is_available() else "cpu"
model = nn.Sequential(nn.Linear(3, 16), nn.ReLU(),
                      nn.Linear(16, 1)).to(device)
opt = torch.optim.SGD(model.parameters(), lr=0.1)

x = torch.randn(64, 3, device=device) # tensores
y = torch.randn(64, 1, device=device)
loss = nn.functional.mse_loss(model(x), y)
loss.backward() # diferenciación automática
opt.step()     # actualiza los pesos
```

29

Historia de los frameworks



30

El ecosistema de deep learning

Theano



MILA, Universidad de Montreal (grupo de Yoshua Bengio)

Pionero: grafos simbólicos y diferenciación automática.

● Concluido · 2017

Caffe → Caffe2



Yangqing Jia (UC Berkeley) → Facebook

Visión por computadora; Caffe2 para móviles y producción.

● Integrado a PyTorch · 2018

PyTorch



Soumith Chintala y equipo (Facebook AI Research)

Grafo de cómputo dinámico y flexible.

● Activo · PyTorch Foundation

TensorFlow



Jeff Dean, Rajat Monga y Google Brain

Escalable y orientado a producción.

● Activo

MXNet



Tianqi Chen y la comunidad DMLC → Apache

Eficiente y escalable.

● Retirado · 2023

Hoy

Opciones principales

PyTorch, TensorFlow y JAX; Keras 3 funciona sobre los tres.

● Este curso: PyTorch

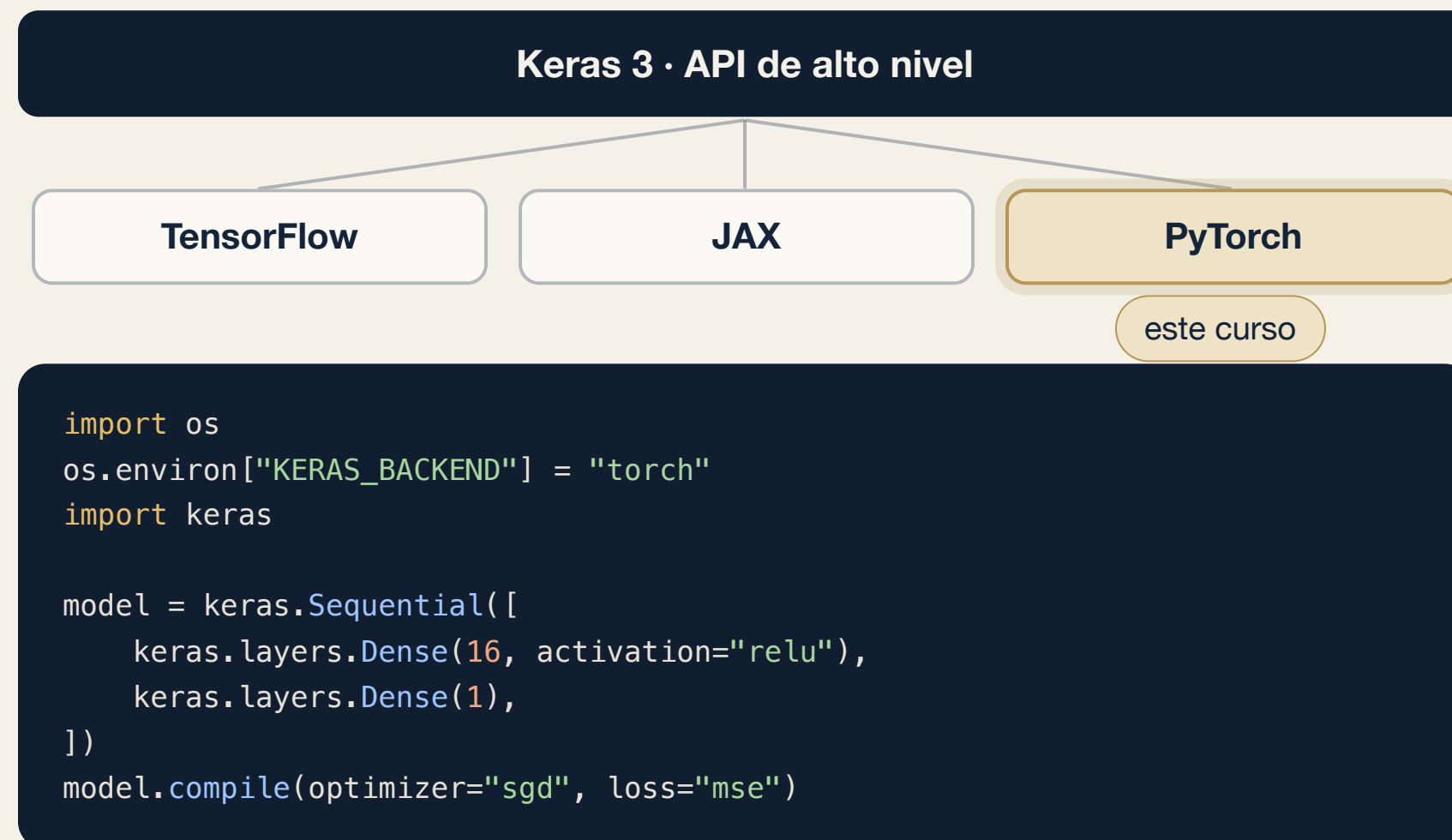
31

Keras

API de alto nivel para definir y entrenar modelos.

Keras 3 admite backends TensorFlow, JAX y PyTorch.

En este curso trabajaremos directamente con PyTorch.



Redes neuronales

¿Cómo ajustamos los parámetros de una red?

02

33

Ingeniería de características

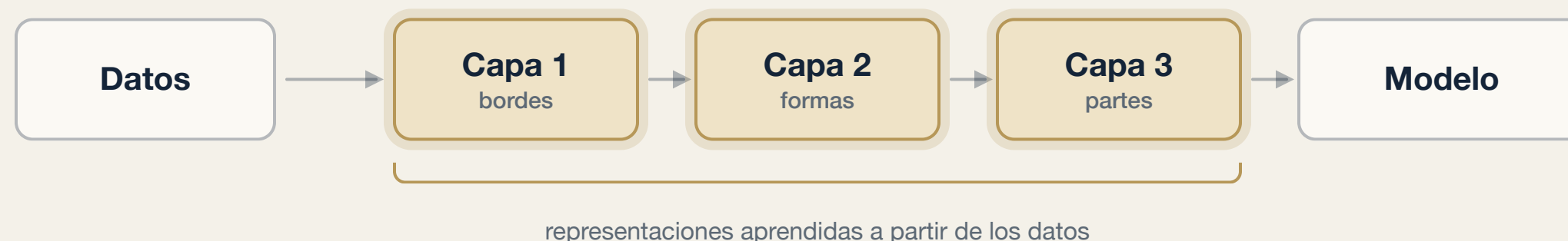
FEATURE ENGINEERING



Feature engineering: diseñar características con conocimiento del problema.

Representation learning: aprender la representación a partir de datos.

REPRESENTATION LEARNING



Las redes profundas pueden aprender varias transformaciones sucesivas.

34

El perceptrón

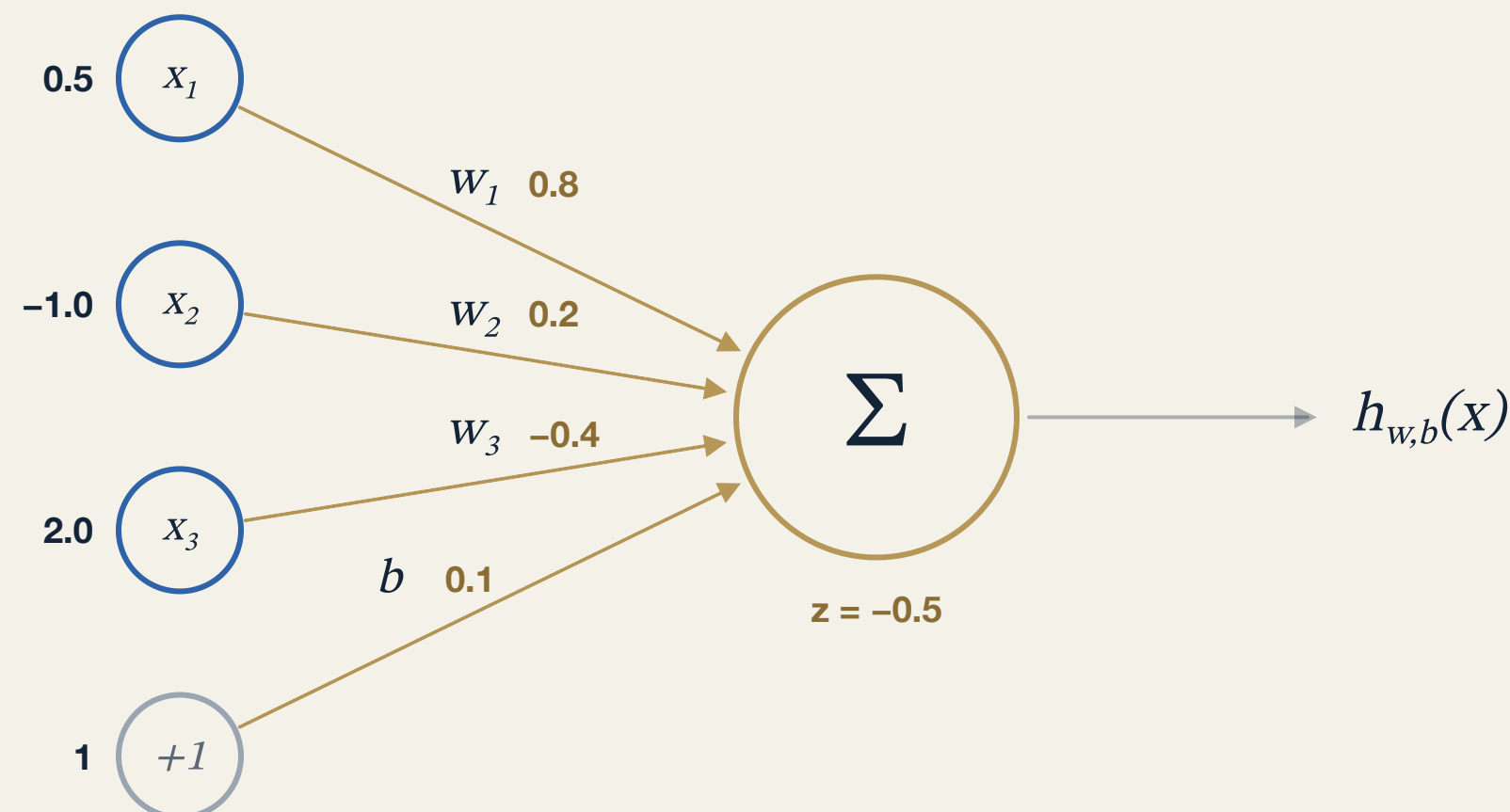
Combina entradas con pesos y un término de sesgo.

$$z = \sum_i w_i x_i + b$$

EJEMPLO

$$z = 0.5 \times 0.8 + (-1.0) \times 0.2 + 2.0 \times (-0.4) + 0.1$$

$$z = 0.4 - 0.2 - 0.8 + 0.1 = -0.5$$



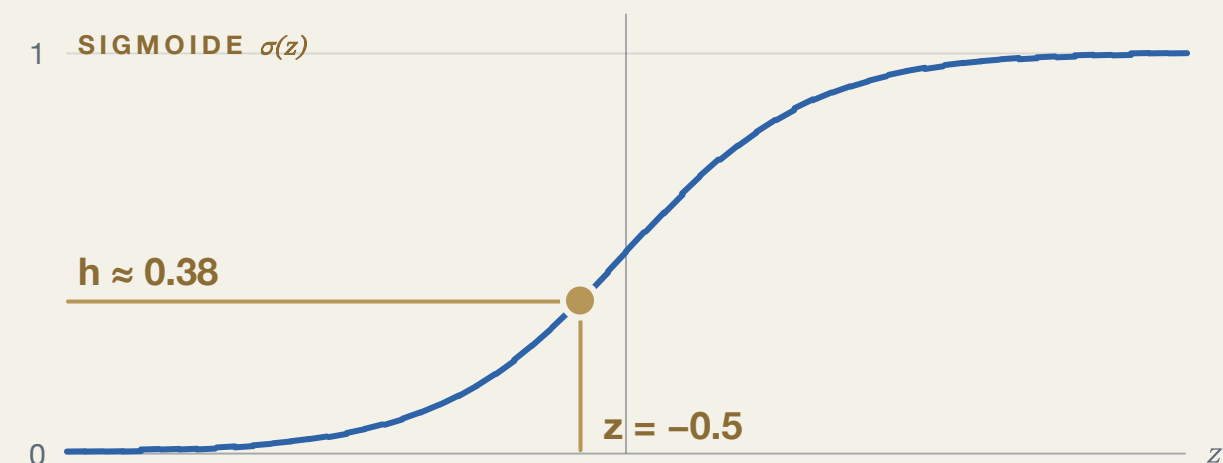
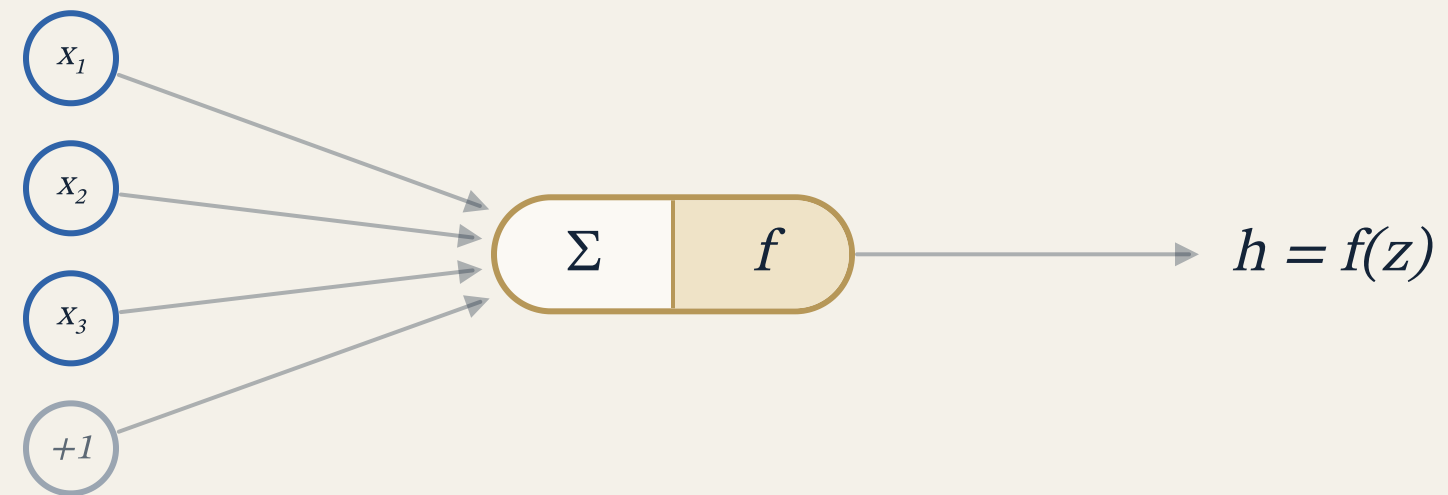
35

La función de activación

La activación transforma la combinación de entradas.

Una función no lineal permite aprender relaciones no lineales.

$$h = f(z)$$



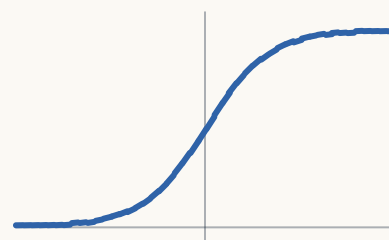
36

Funciones de activación

Sigmoide

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

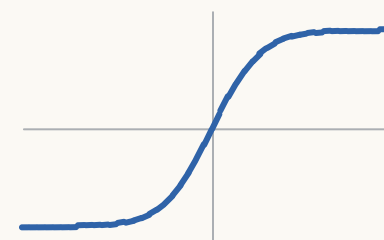
Salida en (0, 1)



tanh

$$\tanh(x)$$

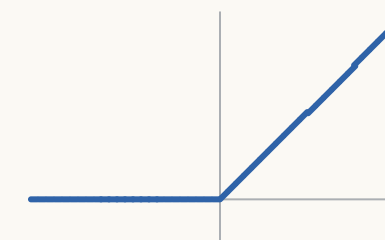
Salida en (-1, 1)



ReLU

$$\max(0, x)$$

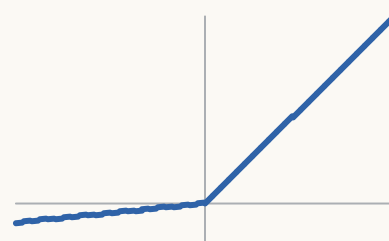
Salida en [0, ∞)



Leaky ReLU

$$\max(0.1x, x)$$

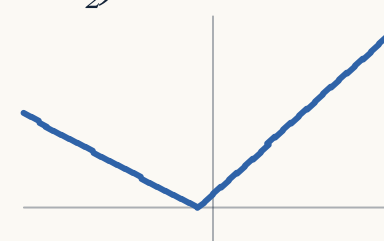
Pendiente 0.1 si $x < 0$



Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

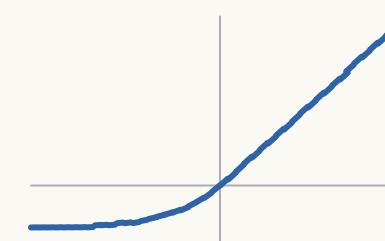
Aprende la forma (ejemplo)



ELU

$$\begin{cases} x, & x \geq 0 \\ \alpha(e^x - 1), & x < 0 \end{cases}$$

Suave si $x < 0 \cdot \alpha = 1$



La elección afecta la señal y los gradientes.

37

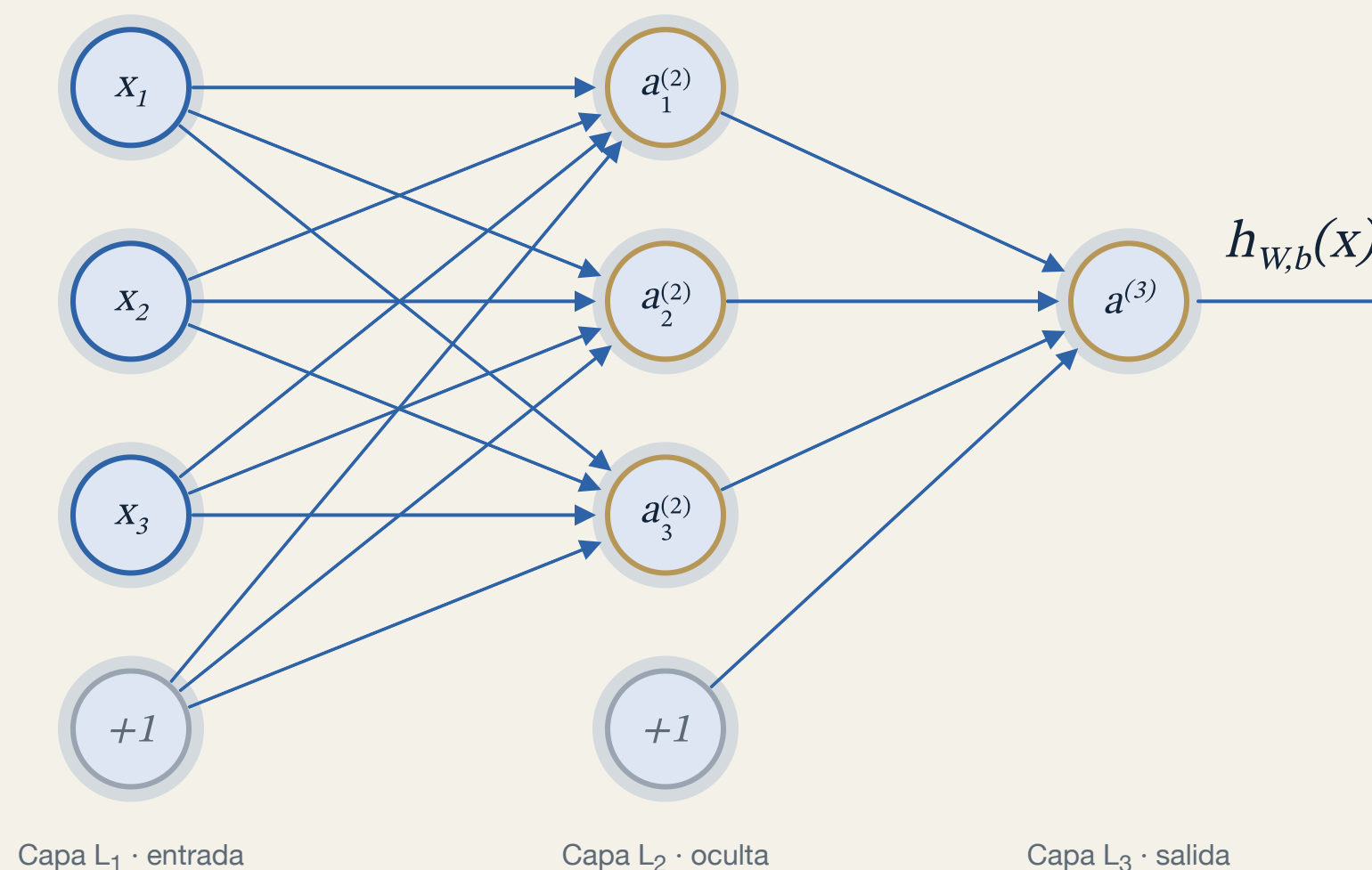
Una red de neuronas

Las salidas de una capa alimentan a la siguiente.

Cada capa transforma la representación.

$$a^{(2)} = f(W^{(1)}x + b^{(1)})$$

$$h_{w,b}(x) = f(W^{(2)}a^{(2)} + b^{(2)})$$



38

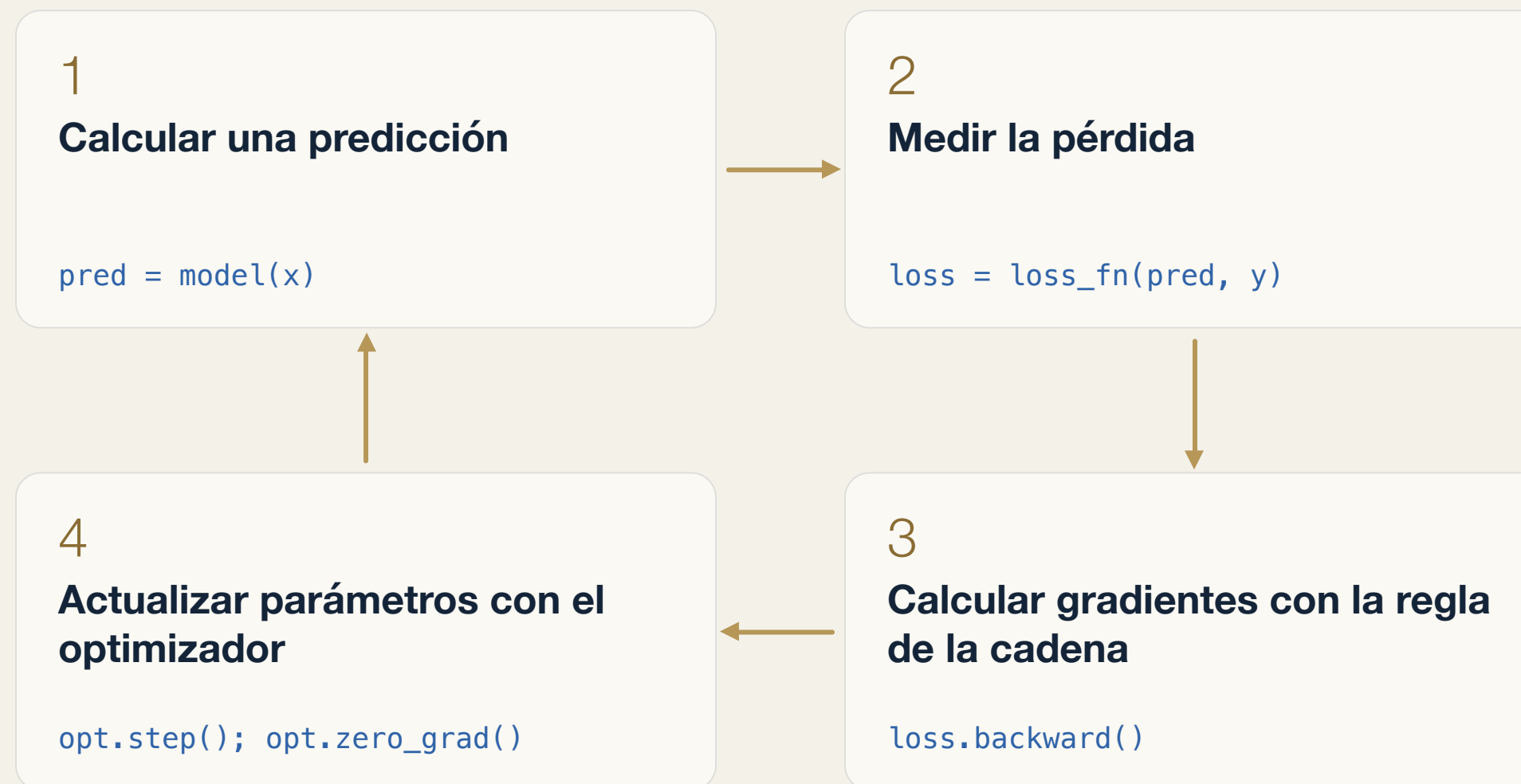
Backpropagation

¿Cómo contribuye cada parámetro al error?

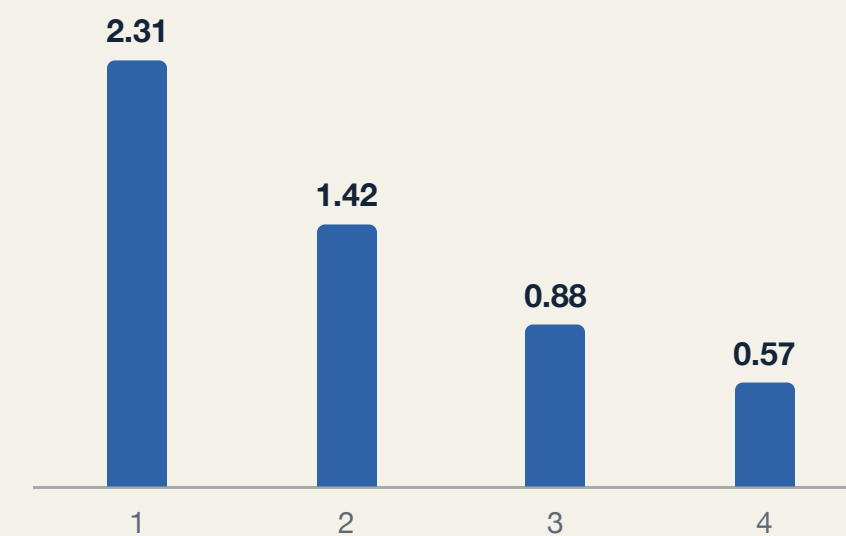
03

39

El ciclo de entrenamiento



PÉRDIDA POR ITERACIÓN (ILUSTRATIVA)
iteración 4



40

El gradiente en cada capa

Backpropagation calcula derivadas de la pérdida.

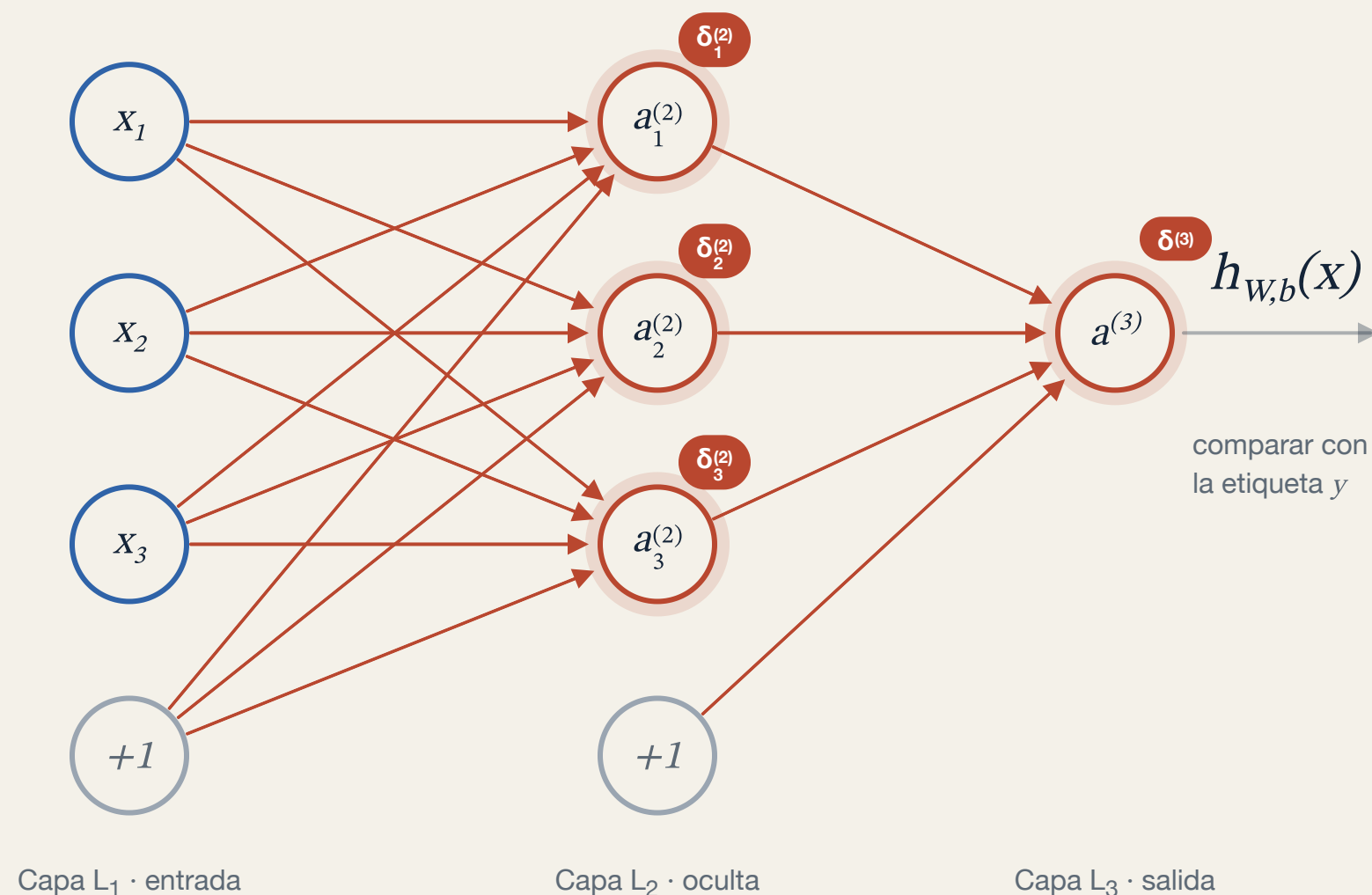
δ representa una señal de error local.

El optimizador usa esos gradientes para actualizar los pesos.

$$\delta^{(2)} = ((W^{(2)})^T \delta^{(3)}) \odot f'(z^{(2)})$$

$$\partial J / \partial W^{(l)} = \delta^{(l+1)} (a^{(l)})^T$$

$$W \leftarrow W - \eta \partial J / \partial W$$



41

Una analogía organizacional

El resultado depende de decisiones en distintos niveles.

La analogía ayuda a pensar en contribuciones locales.

En una red, la contribución se calcula con derivadas.



42

Gradientes que se desvanecen

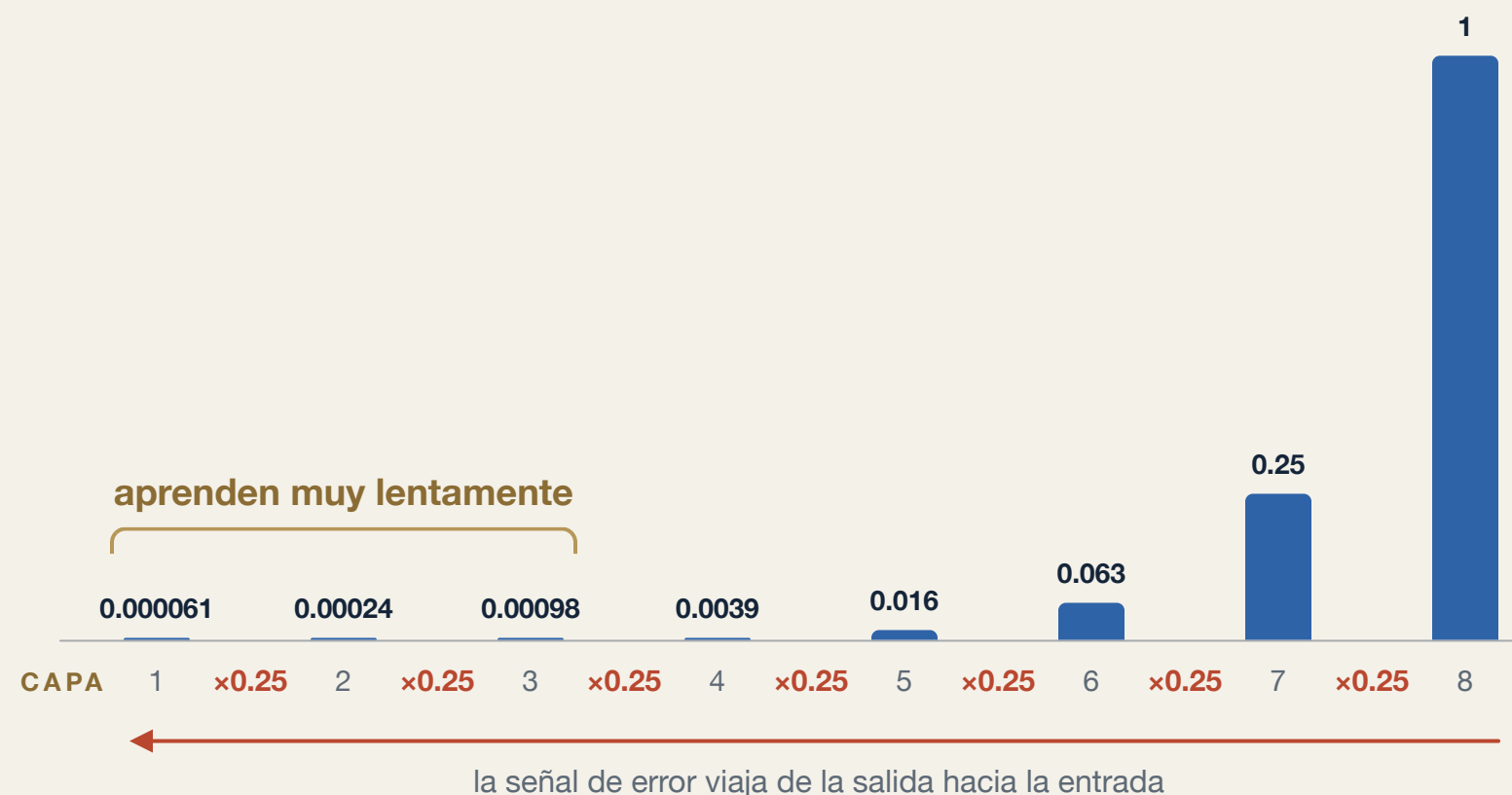
La multiplicación de derivadas pequeñas puede reducir el gradiente.

Las primeras capas pueden aprender muy lentamente.

Activaciones, inicialización y conexiones residuales ayudan a mitigarlo.

El problema puede persistir: no desapareció por completo.

MAGNITUD DEL GRADIENTE POR CAPA · SIGMOIDE: $f'(z) \leq 0.25$, PESOS ≈ 1



ReLU y variantes

Inicialización cuidadosa

Conexiones residuales

43

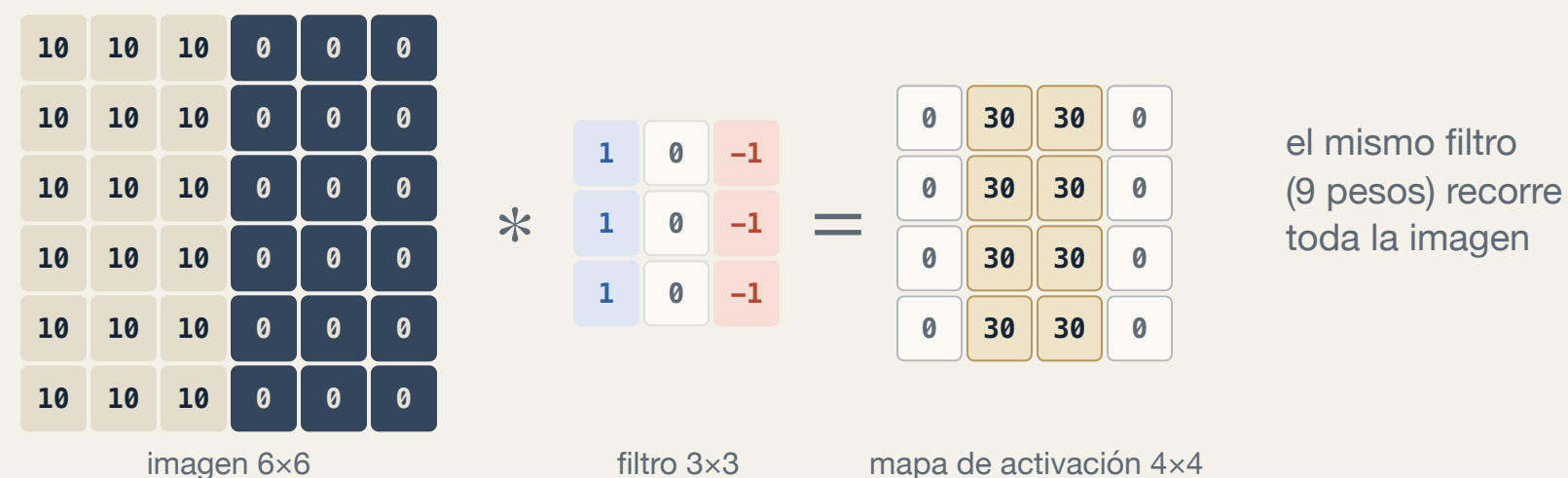
Redes convolucionales

Filtros que detectan patrones locales en imágenes.

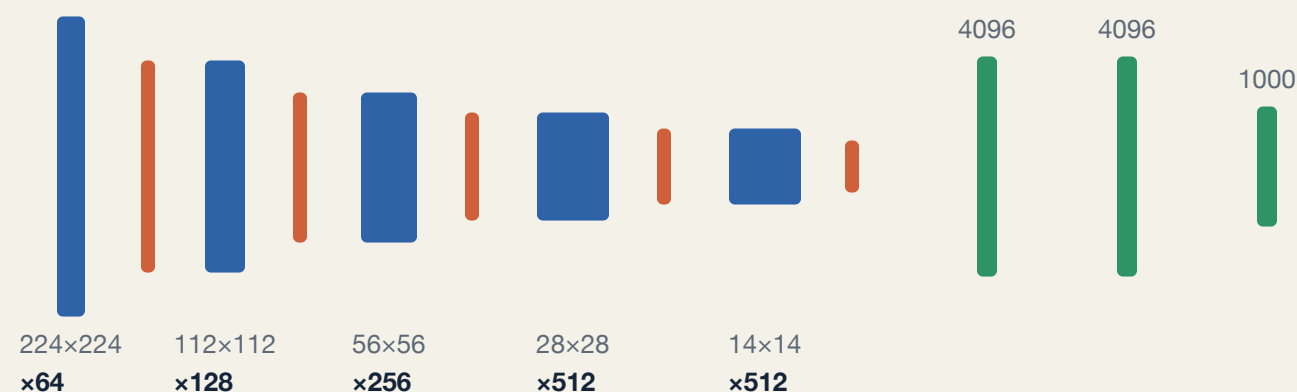
Comparten pesos y aprovechan la estructura espacial.

Una arquitectura importante para visión.

- convolución + ReLU
- max pooling
- totalmente conectada



VGG-16: 13 CAPAS CONVOLUCIONALES + 3 TOTALMENTE CONECTADAS



44

Redes recurrentes

Procesan secuencias mediante un estado que se actualiza.

Aplicaciones en series de tiempo y otros datos secuenciales.

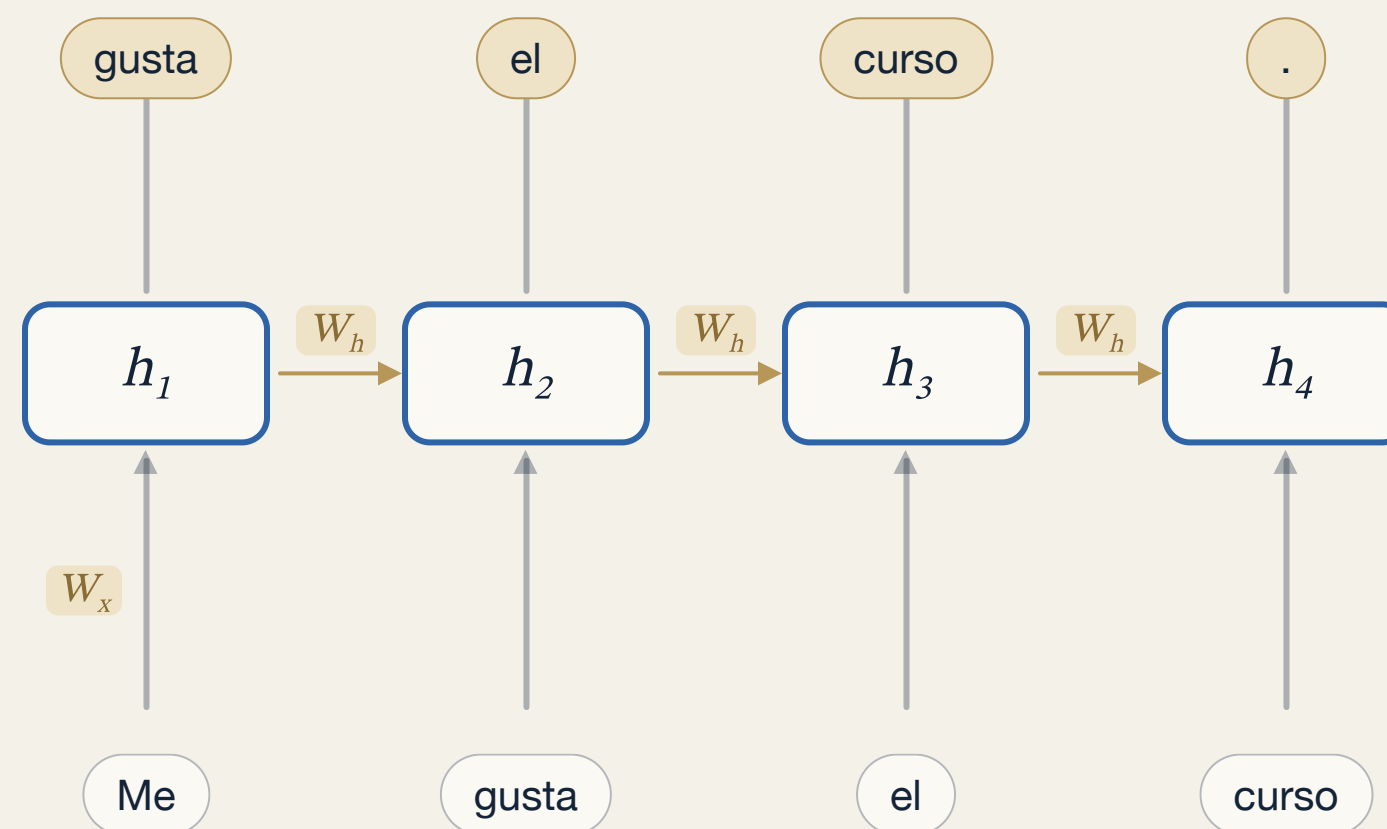
Los Transformers ocupan un papel central en el lenguaje actual.

series de tiempo

texto

audio

sensores



$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$